

This can be demonstrated by a couple of simple instructions:

```
$ psql -U enrico forumdb

forumdb=> SELECT * FROM forum.users;
 pk | username |   gecos   |      email
----+-----+-----+-----
  1 | fluca1978 | Luca Ferrari | fluca1978@gmail.com
  2 | sscotty71 | Enrico Pirozzi | sscotty71@gmail.com
(2 rows)

forumdb=> UPDATE forum.users SET gecos = upper( gecos );
UPDATE 2

forumdb=> SELECT * FROM forum.users;
 pk | username |   gecos   |      email
----+-----+-----+-----
  1 | fluca1978 | LUCA FERRARI | fluca1978@gmail.com
  2 | sscotty71 | ENRICO PIROZZI | sscotty71@gmail.com
(2 rows)
```

As you can see, the user enrico has actually changed the name and surname of the existing users to a full uppercase string. Let's now see what the other user can do:

```
$ psql -U luca forumdb

forumdb=> SELECT * FROM forum.users;
ERROR:  permission denied for table forum.users
forumdb=> SELECT username, gecos FROM forum.users;
 username |   gecos
-----+-----
 fluca1978 | LUCA FERRARI
 sscotty71 | ENRICO PIROZZI
(2 rows)

forumdb=> UPDATE forum.users SET gecos = lower( gecos );
ERROR:  permission denied for table forum.users
```

As you can see, the user `luca` cannot perform anything other than the permissions granted to the `forum_stats` role, that is, a group they belongs to.

It is possible to change the privileges of the user `luca` by either assigning the new grants to the role or by adding another group with more privileges. For instance, if we want all the users in the `forum_stats` group to not be able to read anything other than the columns `username` and `gecos`, while providing `luca` a special grant even if he belongs to that group, it is possible to explicitly set the permission to `luca`, and to him alone:

```
forumdb=# GRANT SELECT ON forum.users TO luca;
GRANT
```

Once the permission has been granted, the `luca` role can use it:

```
% psql -U luca forumdb

forumdb=> SELECT * FROM forum.users;
 pk | username |   gecos   |      email
-----+-----+-----+-----
  1 | fluca1978 | LUCA FERRARI | fluca1978@gmail.com
  2 | sscotty71 | ENRICO PIROZZI | sscotty71@gmail.com
(2 rows)
```

As you can see, the special permission granted to `luca` wins out against the more restrictive one granted to the `forum_stats` group, of which `luca` is a member.

In order to be able to configure your users and groups, you need to understand the privilege chain.

Understanding how privileges are resolved

When a role performs a SQL statement, PostgreSQL checks whether such a role is allowed to perform the task against the object. For example, when the user `luca` performs `SELECT` against the table `users`, PostgreSQL verifies whether the role has been granted permission to do so or not.

If the role has not been granted explicitly (i.e., by means of a `GRANT` statement), PostgreSQL searches for all the groups the role belongs to. If one of the groups has the permission requested, the operation is allowed. If no group has the requested permission, and the permission has not been set for the `PUBLIC` catch-all special role, the operation is rejected.

However, this is only a part of the story: when the system checks the groups a role belongs to, it stops searching for permission if the containing role has been granted to the current one without the INHERIT property. In fact, the GRANT statements allow for the WITH INHERIT clause (that is optional and must contain a true or false value). If the clause is specified and the value is true, the role will dynamically inherit the permissions from the containing group, otherwise, it will not. If the option is missing, the system will use the value of the role INHERIT property: if the role has a NOINHERIT property, the GRANT will implicitly use a WITH INHERIT false clause, otherwise it will implicitly use a WITH INHERIT true clause. Therefore, it is always possible to manage how permissions will be resolved by means of the GRANT statements.

Prior to PostgreSQL 16 the GRANT statement did not support the WITH INHERIT clause, therefore, changing a role's INHERIT property was the only one way to decide how to propagate permissions.

In the previous section, you saw the INHERITS default behavior in action. The luca role inherited permissions that allowed it to perform SELECT of only two columns on the users table. Even if the luca role is not granted that permission, the system checks all the groups they belong to in order to find one, and it finds it in the forums_stats group. Since the permissions are dynamically propagated from a role to all its contained ones, that is, from a group to its members, this is like luca having such permission set on their own role, and so the operation is allowed. Therefore, creating a role with the INHERITS (default) property means that all the permissions granted to such a role will be dynamically propagated to all contained roles.

In order to have a better understanding of this, let's introduce another group, named forum_emails, that can read the email column on the users table, and assign such a group to forum_stats. We would expect that forum_stats, being a member of forum_emails, can read the email column, but since the forum_emails group has been created with the NOINHERIT property, it cannot:

```
-- remove any explicit SELECT permission
-- so luca will have only those from its group
forumdb=# REVOKE SELECT ON forum.users FROM luca;
REVOKE

-- create the new group
forumdb=# CREATE ROLE forum_emails
          WITH NOLOGIN NOINHERIT;
CREATE ROLE

forumdb=# GRANT USAGE ON SCHEMA forum TO forum_emails;
```

```

GRANT

-- assign permissions
forumdb=# GRANT SELECT (email)
          ON forum.users TO forum_emails;

GRANT

-- assign the role to the group
-- implicitly uses WITH INHERIT false
forumdb=# GRANT forum_emails TO forum_stats;
GRANT ROLE

```

Now, luca is a member of forum_stats and forum_emails, but since the latter does not dynamically propagate its permissions to its members, luca cannot get the permissions to read the email column:

```

% psql -U luca forumdb

forumdb=> SELECT username, gecost, email FROM forum.users;
ERROR:  permission denied for table forum.users

```

However, being a member of another role means that a role can always explicitly *become* a group itself, impersonating the latter, and therefore gaining all the permissions granted to the group. This is like the former role is acting on behalf of the group role. In order to be able to act on behalf of the containing group, a role must issue an explicit SET ROLE statement. Clearly, a role cannot become any other arbitrary role: it can act on behalf of explicitly granted roles, and only on behalf of containing groups. Therefore, if the user luca performs an explicit SET ROLE to become the forum_emails role, it will be able to query the email column in the users table:

```

forumdb=> SELECT current_role;
 current_role
-----
 luca
(1 row)

forumdb=> SET ROLE TO forum_emails;
SET

forumdb=> SELECT current_role;

```

```

current_role
-----
forum_emails
(1 row)

forumdb=> SELECT email FROM forum.users;
      email
-----
fluca1978@gmail.com
sscotty71@gmail.com
(2 rows)

forumdb=> SELECT gecost FROM forum.users;
ERROR:  permission denied for table users

```

Let's now change the INHERIT property of the GRANT so that permissions are dynamically propagate::

```

forumdb=# GRANT forum_emails TO forum_stats
        WITH INHERIT true;

GRANT ROLE

```

And now let's see whether the luca role can use both privileges of the forum_stats and forum_emails groups simultaneously:

```

$ psql -U luca forumdb

forumdb=> SELECT gecost, username, email FROM forum.users;
      gecost      | username |      email
-----+-----+-----
LUCA FERRARI      | fluca1978 | fluca1978@gmail.com
ENRICO PIROZZI    | sscotty71 | sscotty71@gmail.com
(2 rows)

```

Great! Now the role can use both group privileges at the very same time without having to explicitly change its current role.

Role inheritance overview

When a role is a member of one or more other roles, the privileges resolution goes like this:

- If the role has the privilege requested, nothing more is checked and the operation is allowed (for example, an explicit GRANT to the role has been issued).
- If the role does not have the requested privilege, the privilege is searched for in the containing groups (if any). If the privilege is found in one of the groups, and the group has been granted with the INHERIT property (either implicitly or explicitly), the permission is dynamically applied. Otherwise, if the permission is found in any of the parent groups, but it has not been granted with the INHERIT option, the permission is not propagated dynamically (i.e., a SET ROLE must be used).

In any case, the role can always exploit the privileges of a group it belongs to via an explicit SET ROLE statement, which means the INHERIT property is used only to dynamically propagate the privileges, therefore preventing the role from needing to change itself into another role.

It is interesting to note that changing a role via an explicit SET ROLE is a declaration that the user is going to perform a particular task that requires particular privileges.

ACLs

PostgreSQL stores permissions assigned to roles and objects as **ACLs**, and, when needed, it examines the ACLs for a specific role and a database object in order to understand whether the command or query can be performed. In this section, you will learn what ACLs are, how they are stored, and how to interpret them to understand what permissions an ACL provides.

It is important to note that ACLs, and therefore permissions, are strictly tied to the role and the database object, which means that granting a specific permission to an object does not mean that the grantee role will have the same permission within another database, even if an object with the same name and nature exists in that database. For example, permitting a role to run PL/Perl code within a database does not automatically endorse it to run PL/Perl code in other databases.

An ACL is a representation of a group of permissions with the following structure:

```
grantee=flags/grantor
```

Where:

- grantee is the name of the to which the permissions are applied.
- flags is the string representing the permissions.
- grantor is the user who granted the permissions.

Whenever the grantor and grantee have the same name, the role is the owner of the database object.

The flags that can be used in an ACL are those reported in the following table. As you can see, not all the flags apply to all the objects: for example, it does not make sense to have a “delete” permission for a function, and it does not make sense to have an “execute” permission for a table:

Flag	Description	Statements	Applies to
a	Append or insert new data	INSERT	Tables and columns
r	Read or get data	SELECT	Tables, columns, and sequences
w	Write or update data	UPDATE	Tables
d	Delete data	DELETE	Tables
D	Delete all data	TRUNCATE	Tables
C	Create a new object	CREATE	Databases, schemas, and table spaces

c	Connect to a database		Databases
t	Trigger or react to data changes	CREATE TRIGGER	Tables
T	Create temporary objects	CREATE TEMP	Tables
x	Cross-reference between data	FOREIGN KEY	Tables
X	Execute runnable code	CALL and SELECT	Functions, routines, and procedures
U	Use various objects		Sequences, schemas, foreign objects, types, and languages

Table 10.1: ACL flags

With the list of possible flags in mind, it becomes easy to decode an ACL such as the following, which is related to a table object:

```
luca=arw/enrico
```

First of all, identify the roles involved: luca and enrico. luca is the role before the equals sign; hence, it is the role the ACL refers to, which means this ACL describes what permissions the luca role has. The other role, enrico, is after the slash sign and therefore is the role that granted the permissions to the luca role. Now, with respect to the flags, the ACL provides append (a), read (r), and write (w) permissions. The above reads as “enrico granted luca the permission to perform INSERT, UPDATE, and SELECT on the table.”

Let’s now see an example of ACLs from a table in the database: you can use the special `\dp psql` command to get information about a table:

```
forumdb=> \dp categories
```

			Access privileges	
Schema	Name	Type	Access privileges	Column privileges
Policies				
forum	categories	table	enrico=arwdDxt/forum+luca=arw/forum+d/forum	

(1 row)

The ACLs are clearly reported in the Access privileges column of the command output. The first line of the ACLs makes a statement regarding the owner of the categories table: since the grantee and the grantor are the same (forum), this is the table owner. The table owner, forum, has all the permissions: append (a), read (r), write (w), delete (d), truncate (D), trigger (t), and cross-reference (x). Therefore, it is possible to read this as “a table owner can do everything on that table.”

The second line of the ACL is the one decoded above, and reads as “luca can INSERT, UPDATE, and SELECT data.” The third line of the ACL is a little more obscure: the grantor is still the forum role, but there is no grantee before the equals sign. This means that the ACL refers to every role. Since the ACL includes only the delete (d) permission, this means that every role in the database can delete rows from the table, as the user forum desires.

ACLs are processed to find a match. Imagine that the luca role wants to delete a row from the table, and therefore issues a DELETE statement. Is that statement allowed or rejected?

Reading the ACL related to the luca role (luca=arw/forum), it is clear that the role cannot delete anything from the table. However, there is a “catch-all” ACL that allows every role to perform a DELETE operation (=d/forum); hence, even the luca role is allowed to remove tuples.

On the other hand, a different role (for example, forum_stats) is not allowed to perform any INSERT on the table because there is no specific permission either for that role or for any other role not explicitly indicated.

But how are those ACLs being produced? First of all, they have all been created by the user enrico, so assuming he is the one connected to the database, the sequence of GRANT statements should have been as follows:

```
-- generates ACL: luca=arw/forum
forumdb=> GRANT SELECT, UPDATE, INSERT
          ON forum categories
          TO luca;

GRANT

-- generates ACL: =d/forum
forumdb=> GRANT DELETE ON forum categories
          TO PUBLIC;

GRANT
```

Now that you have seen how PostgreSQL manages ACLs and how it translates GRANT and REVOKE commands into ACLs, it is time to see what the default permissions are that are granted to a role.

Default ACLs

What happens if an object is created and neither GRANT nor REVOKE is applied to it? The system does not store an ACL for such an object, as you can see by creating a simple empty table and inspecting its privileges:

```
forumdb=> CREATE TABLE perm_test( t text );
CREATE TABLE

forumdb=> \dp perm_test

                                Access privileges
Schema | Name      | Type | Access privileges | Column privileges |
Policies
```